Thread Synchronization Architecture

The Adaptive Smart Cabin & Vehicle Health Monitoring System is a highly concurrent, real-time C++17 application. It leverages multiple background threads running at different frequencies to simulate sensors, evaluate alert strategies, flush logs, and render a live dashboard.

To prevent data races, deadlocks, and performance bottlenecks, the system employs several advanced C++ thread synchronization paradigms.

1. The Threading Model Overview

The system is orchestrated by the `ThreadManager`, which spawns and manages four primary worker threads:

Thread	Frequency	Role	Lock Requirement
Sensor Thread	500 ms	Simulates 6 physical vehicle sensors and updates telemetry.	**Write Lock** (Exclusive)
Monitor Thread	1000 ms	Evaluates sensor data against `AlertRule` strategies.	**Write Lock** (Exclusive)
Logger Thread	1000 ms	Drains queues and flushes asynchronous logs to disk.	**Lock-Free Queueing**
Dashboard Thread	2000 ms	Clears the console and renders the full UI dashboard.	**Read Lock** (Shared)

All centralized state (the sensors array, alert manager, stats, etc.) is held in a single struct called `SharedVehicleData`. All threads receive a `std::shared_ptr<SharedVehicleData>`, ensuring the memory remains valid as long as any thread is alive.

2. Reader-Writer Lock (`std::shared_mutex`)

The core synchronization mechanism protecting `SharedVehicleData` is a `std::shared_mutex` (often called a Reader-Writer lock), named `dataMutex`.

Unlike a standard `std::mutex` which locks out every other thread regardless of what they are doing, a `std::shared_mutex` distinguishes between *reading* and *writing*.

Exclusive Write Access (`std::unique_lock`)

The **Sensor Thread** and **Monitor Thread** must mutate the state (e.g., updating sensor values, adding new alerts to the deque, modifying the historical speed graphs). They acquire the lock like this:

```
{
    std::unique_lock<std::shared_mutex> lock(data_->dataMutex);
    // ... update sensors, push to history vectors ...
}
```

When a `unique_lock` is held, *no other thread* can read or write.

Shared Read Access (`std::shared_lock`)

The **Dashboard Thread** purely reads data to render the screen; it never modifies the sensors or alerts. It acquires the lock like this:

```
{
    std::shared_lock<std::shared_mutex> lock(data_->dataMutex);
    // ... render dashboard safely ...
}
```

A `shared_lock` allows multiple reader threads to access the data concurrently. While this project only has one dashboard thread, this design explicitly models the architectural intent: rendering is a non-destructive read operation.

Scope-Bound Locking

Locks are strictly scoped using braces `{ ... }` so they release *before* the thread calls `std::this_thread::sleep_for()`. Sleeping while holding a lock is a fatal anti-pattern that would cause massive latency spikes across the entire vehicle bus.

3. Producer-Consumer Queue (`SafeQueue<T>`)

Disk I/O is notoriously slow. If the Sensor or Monitor threads wrote directly to the log file while holding the `dataMutex`, the entire simulation would stutter every time the hard drive lagged.

To decouple disk I/O, the architecture implements a **Thread-Safe Producer-Consumer Queue** named `SafeQueue<std::string>`.

The Implementation

`SafeQueue` encapsulates a standard `std::queue` wrapped with its own internal `std::mutex` and a `std::condition_variable`.

- **Producers** (Sensor/Monitor threads) call `push()`. This locks the queue's internal mutex, enqueues the string, and signals the `condition_variable`.
- **Consumers** (Logger thread) call `tryPop()` or `waitPop()`.

Asynchronous Data Flow

When an alert fires, the Monitor thread formats the string and pushes it to `logQueue`. It then immediately releases the `dataMutex` and goes back to sleep. The **Logger Thread** wakes up, drains the `logQueue`, and performs the heavy `stream_.flush()` to the physical disk—completely independent of the simulation critical path.

4. Lock-Free Atomic Shutdown Flag (`std::atomic<bool>`)

To orchestrate a clean, graceful shutdown, the system avoids using heavy mutexes for simple state flags.

Instead, `SharedVehicleData` contains:

```
std::atomic<bool> running{true};
```

Every worker thread's infinite loop is guarded by this atomic variable:

```
while (data_->running.load()) {
    // ...
}
```

When the user triggers a shutdown, `ThreadManager::stop()` simply executes:

```
data_->running.store(false);
```

Because it is `std::atomic`, the CPU guarantees that all threads instantly see the updated value without needing a mutex, preventing data races and memory visibility bugs.

5. RAII (Resource Acquisition Is Initialization)

The entire synchronization model relies heavily on C++ RAII principles:

1. **Lock Guards**: By using `std::lock_guard`, `std::unique_lock`, and `std::shared_lock`, mutexes are automatically released when the lock object goes out of scope. If an exception is thrown inside a critical section, the stack unwinds and the lock is still safely released, preventing deadlocks.
2. **Thread Joining**: The `ThreadManager` destructor automatically calls `stop()`, which sets the `running` flag to false and invokes `.join()` on every thread in the vector. This guarantees that the program cannot exit while background threads are halfway through a critical section.